

Python
for DA

Summary Session 11



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Что такое машинное обучение?



Машинное обучение

область искусственного интеллекта (ИИ), которая занимается разработкой алгоритмов, позволяющих компьютерам обучаться и принимать решения на основе данных. В отличие от традиционного программирования, где компьютеру даются явные инструкции, машинное обучение позволяет системе изучать закономерности и делать прогнозы

Основные определения



Алгоритмы

Это математические модели, которые обрабатывают данные, чтобы найти закономерности и сделать предсказания. К распространенным алгоритмам относятся линейная регрессия, деревья решений и нейронные сети.

Обучающие данные

Это набор данных, используемый для обучения модели машинного обучения. Он включает в себя входные данные и соответствующие выходные метки.

Основные определения



Модель

Модель - это результат работы алгоритма машинного обучения после его обучения на данных. Она может делать прогнозы или принимать решения на основе новых входных данных.

Характеристики

Это отдельные измеряемые свойства или характеристики данных, используемые в модели.

Основные определения



Метки

В обучении с учителем метки - это выходные данные, которые модель обучается предсказывать.

Обучение

Процесс подачи данных в алгоритм машинного обучения, чтобы помочь ему изучить закономерности и взаимосвязи.

Основные определения



Оценка

Оценка эффективности модели с помощью таких показателей, как точность, precision, recall и F1-score.

Оверфиттинг и андерфиттинг

Оверфиттинг происходит, когда модель слишком хорошо усваивает обучающие данные, включая шум и провалы, что делает ее менее эффективной на новых данных. Андерфиттинг происходит, когда модель слишком проста, чтобы уловить основные закономерности в данных.

Экспресс-опрос



В чем различия между машинным обучением и традиционным программированием?

Различие машинного обучения и программирования

Машинное обучение

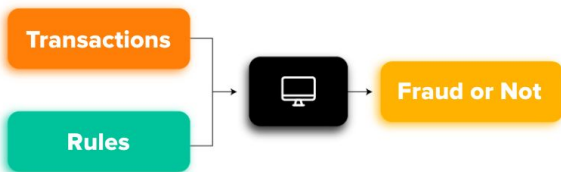
- Управляемое данными
- Вероятностное
- Автоматизированное обучение
- Неструктурированные данные

Традиционное программирование

- Основанное на правилах
- Детерминированное
- Ручное обновление
- Структурированные данные

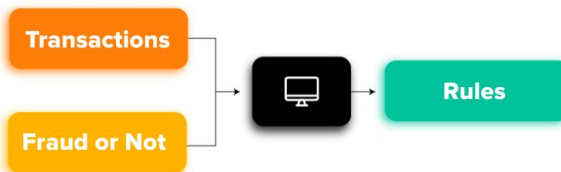
Пример

TRADITIONAL PROGRAMMING



Rule1. Claim time - Submit time < 1 h
Rule2. Agreement review time > 5 m
Rule3. ...

MACHINE LEARNING





Обучение с учителем

Обучение с учителем предполагает обучение модели на наборе данных с метками, где каждая точка данных имеет соответствующую метку или выходное значение. Алгоритм учится сопоставлять входные данные с желаемым выходом, что позволяет ему делать прогнозы для новых, еще невидимых данных.

Ключевые понятия



Маркированные данные



Процесс обучения



Области применения: классификация и регрессия

Распространенные алгоритмы

Классификация

Машины опорных векторов (SVM), логистическая регрессия, деревья решений, Naive Bayes.

Регрессия

Линейная регрессия, полиномиальная регрессия, гребневая регрессия.

Пример



Предсказание того, является ли письмо спамом или нет, на основе таких характеристик, как адрес отправителя, содержание письма и тема письма. Модель обучается на наборе данных писем, помеченных как "спам" или "не спам".



Обучение без учителя

Обучение без учителя предполагает обучение модели на наборе данных без помеченных ответов. Алгоритм пытается найти скрытые закономерности или внутренние структуры в исходных данных.

Ключевые понятия



Немаркированные данные



Обнаружение закономерностей



Области применения: кластеризация и ассоциация

Распространенные алгоритмы

Кластеризация

K-Means Clustering, Hierarchical Clustering, DBSCAN.

Снижение размерности

Анализ главных компонент (PCA), автокодировщики.

Пример



Сегментирование клиентов на различные группы на основе покупательского поведения. Модель анализирует данные, чтобы найти естественные группы клиентов с похожими моделями покупок.



Обучение с подкреплением

Обучение с подкреплением подразумевает обучение агента принимать последовательность решений, взаимодействуя с окружающей средой. Агент учится достигать цели, получая вознаграждения или штрафы в зависимости от своих действий.

Ключевые понятия

-  Агент и среда
-  Вознаграждения и наказания
-  Пробы и ошибки
-  Области применения

Распространенные алгоритмы

Q-Learning

Метод, основанный на ценности, в котором агент узнает ценность действий в состояниях.

SARSA (State-Action-Reward- State-Action)

Аналогичен Q-Learning, но обновляет функцию "действие-ценность" в зависимости от выполненного действия.

Глубокие Q-сети (DQN)

Сочетает Q-Learning с глубокими нейронными сетями для работы с высокоразмерными пространствами состояний.

Пример



Обучение робота перемещению по лабиринту. Робот получает вознаграждение за достижение конца лабиринта и штрафы за столкновение со стенами. Со временем он учится оптимальному пути к цели.



Разделение данных на обучающие и тестовые наборы

Фундаментальная практика машинного обучения, которая обеспечивает разработку надежных и достоверных моделей

Основные причины для разделения

-  Оценка модели и оценка производительности
-  Предотвращение оверфиттинга
-  Обеспечение беспристрастной оценки
-  Настройка гиперпараметров и выбор модели
-  Моделирование сценариев реального мира
-  Перекрестная валидация для повышения надежности

Экспресс-опрос



Что такое Scikit-learn?



Scikit-learn

популярная библиотека Python для машинного обучения, предоставляет удобную функцию `train_test_split` для выполнения такого разделения данных.

Шаг 1: Импорт необходимых библиотек

```
from sklearn.model_selection import train_test_split  
from sklearn.datasets import load_iris  
import matplotlib.pyplot as plt
```

Шаг 2: Загрузка набора данных

```
# Загрузить набор данных Iris
iris = load_iris()
X = iris.data # Характеристики
y = iris.target # Метки
```

Шаг 3: Выполните разбиение

```
# Разделить данные на обучающий и тестовый наборы  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```


Шаг 4: Проверка разбиения

```
print(f"Размер обучающих данных (X_train): {X_train.shape}")  
print(f"Размер тестовых данных (X_test): {X_test.shape}")  
print(f"Размер обучающих меток (y_train): {y_train.shape}")  
print(f"Размер тестовых меток (y_test): {y_test.shape}")
```

Пример вывода

Размер обучающих данных (X_train): (120, 4)

Размер тестовых данных (X_test): (30, 4)

Размер обучающих меток (y_train): (120,)

Размер тестовых меток (y_test): (30,)

Шаг 5. Изображение выборки

```
fig, axs = plt.subplots(nrows= 1 , ncols= 3 )

axs[0].boxplot(X)
axs[0].title.set_text("Все данные")
axs[1].boxplot(X_train)
axs[1].title.set_text("Тренировочные")
axs[2].boxplot(X_test)
axs[2].title.set_text("Тестовые")
plt.show()
```



Стратифицированное разделение обучения и тестирования

Для задач классификации часто важно сохранить одинаковое распределение классов в обучающем и тестовом наборах. Этого можно добиться с помощью параметра `stratify`.

Применение параметра stratify

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42, stratify=y)
```



Разделение на обучение-валидацию-тестирование

В некоторых случаях для настройки гиперпараметров полезно иметь отдельный валидационный набор. Это можно сделать, сначала разделив данные на обучающий и временный наборы, а затем разделив временный набор на валидационный и тестовый наборы.

Реализация

```
# Первоначальное разбиение на обучающий и временный наборы
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4,
random_state=42)

# Дальнейшее разбиение временного набора на валидационный и тестовый наборы
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
random_state=42)
```

Визуализация

```
fig, axs = plt.subplots(nrows= 1 , ncols= 4 )

axs[0].boxplot(X)
axs[0].title.set_text("Все данные")
axs[1].boxplot(X_train)
axs[1].title.set_text("Тренировочные")
axs[2].boxplot(X_test)
axs[2].title.set_text("Тестовые")
axs[3].boxplot(X_val)
axs[3].title.set_text("Валидационные")
plt.show()
```




ВОПРОСЫ



Экспресс-опрос

?

Что такое линейная регрессия?



Линейная регрессия

один из самых простых и широко используемых алгоритмов в машинном обучении и статистике. Это метод контролируемого обучения, используемый для прогнозирования непрерывной итоговой переменной (зависимой переменной) на основе одной или нескольких предикторных переменных (независимых переменных)

Цель линейной регрессии



Найти наилучшим образом подходящую прямую линию через точки данных, которая может быть использована для прогнозирования.

Ключевые термины

**Зависимая
переменная (Y)**

Переменная, которую мы
пытаемся предсказать.

**Независимая
переменная (X)**

Переменная(ые),
используемая(ые) для
прогнозирования.

Линия регрессии

Линия наилучшего
соответствия, которая
минимизирует разницу между
фактическими точками
данных и предсказанными
значениями.

Взаимосвязь между переменными

$$y=mx+b,$$

где:

y - прогнозируемое значение зависимой переменной.

x - независимая переменная.

m - наклон линии (насколько y изменяется на единицу изменения в x).

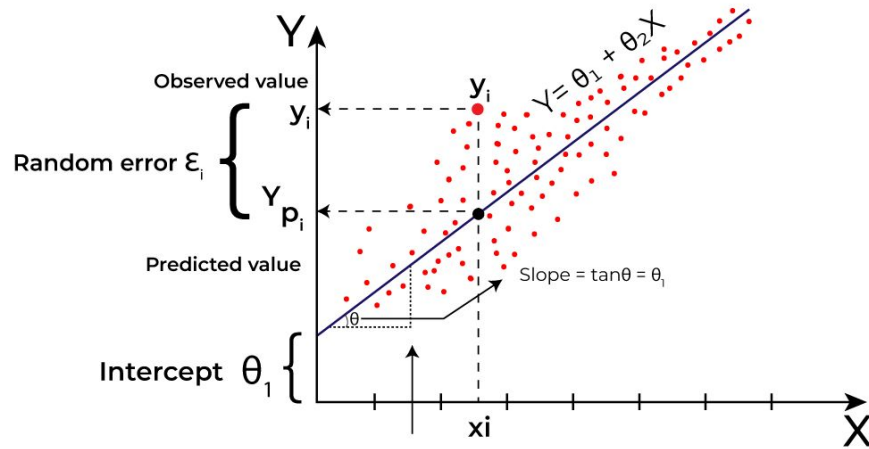
b - y -интерцепт (значение y , когда x равно 0).



Регрессия по методу наименьших квадратов регрессия

Линия наилучшего соответствия определяется путем минимизации суммы квадратов разностей между фактическими и предсказанными значениями

Регрессия по методу наименьших квадратов





Средняя квадратичная ошибка (MSE)

Измеряет среднее значение квадратов ошибок (разницы между фактическими и предсказанными значениями)

Определение MSE

$$J(m, b) = \frac{1}{N} \sum_{i=1}^N (y_i - (mx_i + b))^2,$$

где y_i - фактическое значение, $mx_i + b$ - прогнозируемое значение.

Задача состоит в том, чтобы найти значения m и b , которые минимизируют эту функцию потерь.

Экспресс-опрос

?

Что такое градиентный спуск?



Градиентный спуск

это алгоритм оптимизации, используемый для минимизации функции потерь. Он итеративно регулирует параметры m и b , чтобы найти минимальное значение функции потерь. Этот алгоритм имеет параметр α , который обозначает скорость обучения

Шаг 1: Импорт необходимых библиотек

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Шаг 2: Загрузка набора данных

```
# Создание примера набора данных
data = {
    'Mother': [170.2, 168.9, 162.6, 162.6, 148.6, 172.7],
    'Daughter': [175.3, 166.4, 172.7, 170.2, 158.8, 176.5]
}
df = pd.DataFrame(data)
```

Шаг 3: Изучение данных

```
# Отообразим первые несколько строк набора данных  
print(df.head())
```

```
# Сводная статистика  
print(df.describe())
```

Шаг 4: Разбиение на наборы

```
# Определяем переменные предиктора (X) и отклика (y)
X = df[['Mother']]
y = df['Daughter']

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```


Шаг 5: Обучение модели

```
# Инициализируем модель линейной регрессии
model = LinearRegression()

# Обучаем модель на обучающих данных
model.fit(X_train, y_train)
```

Шаг 6: Оценка модели

```
# Делаем прогнозы на тестовом наборе
y_pred = model.predict(X_test)

# Рассчитайте метрики оценки
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")
```

Шаг 7: Визуализация результатов

```
# Постройте график обучающих данных и линии регрессии
plt.scatter(X_train, y_train, color='blue', label='Обучающие
данные')
plt.scatter(X_test, y_test, color='green', label='Тестовые данные')
plt.plot(X_train, model.predict(X_train), color='red', linewidth=2,
label='Линия регрессии')
plt.xlabel('Рост матери (см)')
plt.ylabel('Рост дочери (см)')
plt.legend()
plt.show()
```

Экспресс-опрос

?

Что такое логистическая регрессия?



Логистическая регрессия

это фундаментальный алгоритм машинного обучения, используемый для решения задач бинарной классификации. В отличие от линейной регрессии, которая предсказывает непрерывные значения, логистическая регрессия предсказывает вероятность того, что заданный входной сигнал принадлежит к одному из двух возможных классов.

Ключевые термины

Бинарная классификация

Логистическая регрессия в основном используется для решения задач бинарной классификации, когда переменная результата может принимать только два возможных значения, например «да» или «нет», «спам» или «не спам», «болезнь» или «не болезнь».

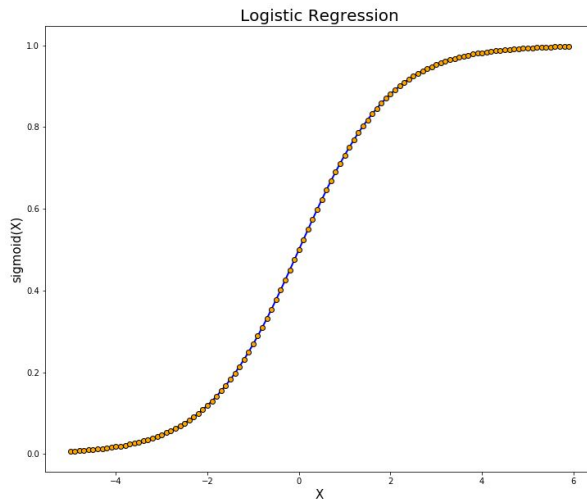
Вероятностное прогнозирование

Цель логистической регрессии - предсказать вероятность того, что данный входной сигнал принадлежит к определенному классу.

Логистическая функция

В основе логистической регрессии лежит логистическая функция, также известная как сигмоидная функция. Эта функция отображает любое вещественное число на значение между 0 и 1, что делает ее подходящей для моделирования вероятностей.

$$\sigma(z) = \frac{1}{1+e^{-z}}.$$



Линейная комбинация входов

В логистической регрессии входными данными для сигмоидальной функции z является линейная комбинация входных признаков. Это похоже на линейное уравнение, используемое в линейной регрессии:

$$z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p,$$

где x_i - входные признаки.

Модель логистической регрессии

Модель логистической регрессии объединяет линейное уравнение с сигмоидальной функцией для прогнозирования вероятности того, что данный вход принадлежит к положительному классу (например, классу 1):

$$P(y = 1|x) = \sigma(z).$$

Это уравнение дает вероятность того, что выход y равна 1 при наличии входных признаков x .

Граница принятия решения

Чтобы принять решение о бинарной классификации, мы устанавливаем порог (обычно 0,5). Если предсказанная вероятность больше или равна порогу, мы относим входные данные к положительному классу (1). В противном случае мы классифицируем его как отрицательный класс (0):

Если $P(y = 1|x) \geq 0.5$, предскажите $y=1$.

Если $P(y = 1|x) < 0.5$, предскажите $y=0$.

Функция потерь и оптимизация

Для обучения модели логистической регрессии нам необходимо найти оптимальные значения коэффициентов $(\beta_0, \beta_1, \dots, \beta_p)$. Это делается путем минимизации функции потерь, которая измеряет, насколько хорошо предсказания модели соответствуют фактическим результатам.

$$J(\beta) = - \frac{1}{N} \sum_{i=1}^N [y_i \log(P(y_i|x_i)) + (1 - y_i) \log(1 - P(y_i|x_i))].$$

Задача состоит в том, чтобы найти коэффициенты β , которые минимизируют эту функцию стоимости. Обычно это делается с помощью алгоритмов оптимизации, таких как градиентный спуск.

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

Шаг 2: Загрузка и подготовка набора данных

```
# Создание примера набора данных
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Pass_Fail': [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
```

Шаг 3: Выполните деление

```
# Определяем переменные-предикторы (X) и переменные-ответы (y)
X = df[['Hours_Studied']]
y = df['Pass_Fail']

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Шаг 4: Обучение модели

```
# Инициализируем модель логистической регрессии
model = LogisticRegression()

# Обучаем модель на обучающих данных
model.fit(X_train, y_train)
```

Шаг 5: Создание прогнозов

```
# Делаем предсказания на тестовом наборе  
y_pred = model.predict(X_test)
```


Шаг 6: Оценка модели

```
# Вычислить точность
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)

# Отчет о классификации
print('Отчет о классификации:')
print(classification_report(y_test, y_pred))
```

Шаг 7: Визуализация результатов

```
# Постройте график точек данных
plt.scatter(X, y, color='blue', label='Data points')

# Построение кривой логистической регрессии
X_range = np.linspace(X.min(), X.max(), 300)
y_prob = model.predict_proba(X_range.reshape(-1, 1))[:, 1]
plt.plot(X_range, y_prob, color='red', linewidth=2, label='Кривая
логистической регрессии')

plt.xlabel('Изученные часы')
plt.ylabel('Вероятность сдачи')
plt.legend()
plt.show()
```

Экспресс-опрос

?

Что такое деревья решений?



Деревья решений

это популярный и интуитивно понятный алгоритм машинного обучения, используемый для решения задач классификации и регрессии. Они называются «деревьями», поскольку используют древовидную модель решений и их возможных последствий, включая случайные исходы событий, затраты ресурсов и полезность.

Ключевые термины

Узлы

Каждый узел представляет собой признак (атрибут) в наборе данных.

Грани

Каждое ребро представляет собой правило принятия решения, основанное на признаке.

Листья

Каждый узел листа представляет результат (метку класса или непрерывное значение).

Разбиение данных



Алгоритм дерева решений работает путем рекурсивного разбиения данных на подмножества на основе значений признаков. Цель состоит в том, чтобы создать подмножества, максимально однородные по отношению к целевой переменной (переменной, которую мы хотим предсказать).

Выбор наилучшего разбиения

Индекс разнообразия Джини

Измеряет вероятность неправильной классификации случайно выбранного элемента, если он был случайно помечен в соответствии с распределением меток в подмножестве.

Энтропия

Измеряет количество неопределенности или случайности в подмножестве.

Информационный выигрыш

Измеряет уменьшение энтропии или примеси после разбиения.

Создание узлов принятия решений и узлов листьев

Узлы принятия решений

Эти узлы представляют признак и условие, используемые для разделения данных. Каждый узел принятия решения имеет две или более ветвей.

Узлы листьев

Эти узлы представляют конечный результат или предсказание. В задачах классификации они представляют метки классов. В задачах регрессии они представляют непрерывные значения.

Рекурсивное разбиение



Процесс разбиения данных и создания узлов принятия решений и узлов листа повторяется рекурсивно для каждого подмножества.

Преимущества деревьев принятия решений



Легко понять и интерпретировать



Работает как с числовыми, так и с категориальными данными



Непараметрические

Недостатки деревьев принятия решений



Склонность к переоценке



Чувствительность к изменениям данных



Предвзятое отношение к признакам с большим количеством уровней

Шаг 1: Импорт необходимых библиотек

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

Шаг 2: Загрузка и подготовка данных

```
# Создание примера набора данных
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Attendance': ['Плохая', 'Плохая', 'Плохая', 'Хорошая',
                  'Хорошая', 'Хорошая', 'Хорошая', 'Хорошая', 'Плохая'],
    'Pass_Fail': [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)

# Преобразуем категориальные данные в числовые
df['Attendance'] = df['Attendance'].map({'Плохая': 0, 'Хорошая':
1})# Создание примера набора данных
```

Шаг 3: Выполните разделения данных

```
# Определяем переменные-предикторы (X) и переменные-ответы (y)
X = df[['Hours_Studied', 'Attendance']]
y = df['Pass_Fail']

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Шаг 4: Обучение модели

```
# Инициализируем классификатор дерева решений  
model = DecisionTreeClassifier(random_state=42)
```

```
# Обучить модель на обучающих данных  
model.fit(X_train, y_train)
```

Шаг 5: Предсказание

```
# Делаем предсказания на тестовом наборе  
y_pred = model.predict(X_test)
```


Шаг 6: Оценка модели

```
# Вычислить точность
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)

# Отчет о классификации
print('Отчет о классификации:')
print(classification_report(y_test, y_pred))
```

Шаг 7: Визуализация

```
# Постройте дерево решений
plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=['Hours_Studied', 'Attendance'],
class_names=['Fail', 'Pass'], filled=True)
plt.show()
```



К ближайших соседей (KNN)

это простой, интуитивно понятный и широко используемый алгоритм машинного обучения для задач классификации и регрессии. Это тип обучения на основе экземпляров, когда алгоритм делает прогнозы на основе сходства между входными данными и данными обучения

Ключевые термины

Обучение на основе экземпляров

KNN - это алгоритм обучения на основе экземпляров, что означает, что он запоминает обучающий набор данных и делает прогнозы на основе сохраненных экземпляров.

Ленивое обучение

KNN считается ленивым алгоритмом обучения, поскольку он не строит явную модель на этапе обучения. Вместо этого он откладывает вычисления до получения запроса.

Мера сходства

KNN использует метрику расстояния (например, евклидово расстояние) для измерения сходства между точками данных.

Выбор количества соседей (K)



Первым шагом в KNN является выбор количества соседей (K). Это гиперпараметр, определяющий, сколько ближайших соседей будет учитываться при составлении прогноза.

Влияние K на производительность алгоритма

- Малое значение K: небольшое значение K (например, $K=1$) может привести к созданию модели, чувствительной к шуму в данных, что может вызвать перебор.
- Большое значение K. Большое значение K может сгладить предсказания, но также может привести к недостаточной подгонке, когда модель не сможет уловить важные закономерности в данных.

Вычисление расстояния



$$\text{Евклидово расстояние} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2},$$

где x_i и y_i - значения признаков для точки входных данных и точки обучающих данных, соответственно, n - количество признаков.

Поиск ближайших соседей



После вычисления расстояний KNN определяет K ближайших соседей точки входных данных. Это K точек данных в обучающем наборе данных, которые имеют наименьшее расстояние до точки входных данных

Составление прогнозов

- Классификация: Для задач классификации KNN присваивает метку класса, которая является наиболее распространенной среди K ближайших соседей. Для этого используется механизм голосования по большинству голосов.
- Регрессия: Для задач регрессии KNN предсказывает значение путем усреднения значений K ближайших соседей.

Преимущества KNN



Простота и интуитивность



Отсутствие фазы обучения



Универсальность

Недостатки KNN



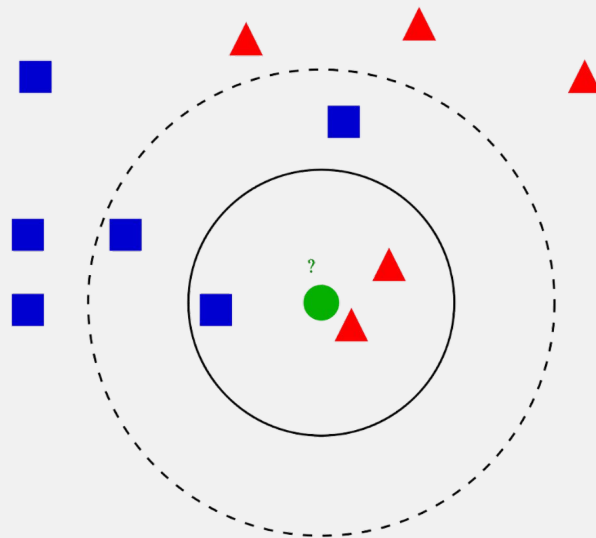
Вычислительные затраты



Чувствительность к нерелевантным признакам



Выбор



Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

Шаг 2: Загрузка и подготовка данных

```
# Создание примера набора данных
data = {
    'Weight': [150, 160, 170, 180, 190, 200, 210, 220, 230, 240],
    'Color_Score': [0.8, 0.85, 0.9, 0.95, 1.0, 0.75, 0.7, 0.65, 0.6,
0.55],
    'Label': [0, 0, 0, 0, 0, 1, 1, 1, 1, 1] # 0 для яблока, 1 для
апельсина
}
df = pd.DataFrame(data)
```

Шаг 3: Визуализация набора данных

```
# Построение графика точек данных
plt.scatter(df['Weight'], df['Color_Score'], c=df['Label'],
            cmap='coolwarm', edgecolors='k')
plt.xlabel('Вес')
plt.ylabel('Цветовой балл')
plt.title('Классификация фруктов')
plt.show()
```

Шаг 4: Выполните разбиение

```
# Определяем переменные предиктора (X) и отклика (y)
X = df[['Weight', 'Color_Score']]
y = df['Label']

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Шаг 5: Обучение KNN-модели

```
# Инициализируем KNN-классификатор
knn = KNeighborsClassifier(n_neighbors=3)

# Обучить модель на обучающих данных
knn.fit(X_train, y_train)
```


Шаг 6: Предсказание

```
# Делаем предсказания на тестовом наборе  
y_pred = knn.predict(X_test)
```

Шаг 7: Оценка модели

```
# Вычисляем точность
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)

# Отчет о классификации
print('Отчет о классификации:')
print(classification_report(y_test, y_pred))
```

Шаг 8: Визуализация

```
# Построение графика границы принятия решения
h = 0.01 # размер шага в сетке
x_min, x_max = X['Weight'].min() - 1, X['Weight'].max() + 1
y_min, y_max = X['Color_Score'].min() - 0.1, X['Color_Score'].max()
+ 0.1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min,
y_max, h))
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X['Weight'], X['Color_Score'], c=y, edgecolors='k',
cmap='coolwarm')
plt.xlabel('Вес')
plt.ylabel('Цветовая оценка')
plt.title('Граница принятия решения для KNN')
plt.show()
```



ВОПРОСЫ



Экспресс-опрос

?

Что такое оценка модели?



Оценка модели

Это шаг в рабочем процессе машинного обучения, который включает в себя оценку производительности обученной модели на отдельном наборе данных, который не использовался в процессе обучения.

Важность оценки эффективности модели

-  Обобщение на невидимые данные
-  Предотвращение избыточной и недостаточной подгонки: оверфиттинг и андерфиттинг
-  Выбор модели
-  Настройка гиперпараметров
-  Метрики производительности
-  Укрепление доверия и уверенности
-  Выявление слабых сторон модели
-  Обеспечение справедливости и выявление предвзятости

Предотвращение избыточной и недостаточной подгонки

Оверфиттинг

Возникает, когда модель слишком хорошо усваивает обучающие данные, включая шумы и выбросы, что приводит к низкой производительности на новых данных. Оценка модели на тестовом наборе помогает выявить оверфиттинг.

Андерфиттинг

Возникает, когда модель слишком проста, чтобы отразить основные закономерности в данных, что приводит к низкой производительности как на обучающих, так и на тестовых данных. Оценка модели помогает выявить андерфиттинг и подсказывает необходимость создания более сложной модели.

Метрики производительности

Меткость
(accuracy)

Доля правильно предсказанных
экземпляров от общего числа
экземпляров

Точность
(precision)

Recall
(чувствительность)

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

Доля верных положительных
предсказаний среди всех
положительных предсказаний

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

Доля истинно положительных
предсказаний от всех истинно
положительных

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Среднее гармоническое значение
между показателями precision и recall,
обеспечивающее баланс между ними

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Среднее значение квадратичной
разницы между прогнозируемыми и
фактическими значениями

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Мера того, насколько хорошо
предсказания модели соответствуют
фактическим значениям

Экспресс-опрос

?

Что такое Accuracy?



Accuracy

Это метрика для оценки эффективности модели классификации. Измеряет долю правильно предсказанных экземпляров (как истинно положительных, так и истинно отрицательных) от общего числа экземпляров в наборе данных. Меткость говорит о том, как часто модель делает правильные прогнозы.

Формула для расчета точности



Accuracy = Количество правильных предсказаний/Общее количество предсказаний

Меткость в терминах компонентов матрицы ошибок

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN},$$

где:

- *TP* (True Positives): Количество экземпляров, правильно предсказанных как положительный класс.
- *TN* (True Negatives): Количество экземпляров, правильно предсказанных как отрицательный класс.
- *FP* (False Positives): Количество экземпляров, неверно предсказанных как положительный класс.
- *FN* (False Negatives): Количество экземпляров, неверно предсказанных как отрицательный класс.

Пример

	Предсказанное положительное	Предсказанное отрицательное
Фактическое положительное	50	10
Фактическое отрицательное	5	35

Пример

Из матрицы ошибок мы можем определить следующие значения:

$$TP=50, TN=35, FP=5, FN=10.$$

Используем формулу точности:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} = \frac{50+35}{50+35+5+10} = \frac{85}{100}$$

Меткость модели составляет 0,85, или 85%.

Accuracy



Преимущества

- Простота
- Интуитивная интерпретация

Ограничения

- Несбалансированные наборы данных
- Отсутствие детализации

Когда использовать Accuracy



Классы в наборе данных сбалансированы, то есть количество экземпляров в каждом классе примерно одинаково.



Стоимость ложноположительных и ложноотрицательных ошибок одинакова, и нет существенной разницы в значимости различных типов ошибок.

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

Шаг 2: Загрузка и подготовка набора данных

```
from sklearn.datasets import load_iris

# Загрузить набор данных
iris = load_iris()
X, y = iris.data, iris.target

# Преобразование в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = y
```


Шаг 3: Разбиение на обучающие и тестовые выборки

```
from sklearn.datasets import load_iris

# Загрузить набор данных
iris = load_iris()
X, y = iris.data, iris.target

# Преобразование в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = y
```

Шаг 4: Обучение модели логистической регрессии

```
# Инициализируем модель логистической регрессии
model = LogisticRegression(max_iter=200)

# Обучаем модель на обучающих данных
model.fit(X_train, y_train)
```

Шаг 5: Делаем предсказания

```
# Делаем предсказания на тестовом наборе  
y_pred = model.predict(X_test)
```

Шаг 6: Оценка модели

```
# Вычислить точность
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy}')
```

Шаг 7: Отображение матрицы ошибок

```
# Отображение матрицы ошибок
conf_matrix = confusion_matrix(y_test, y_pred)
print('Матрица ошибок:')
print(conf_matrix)

# Визуализация матрицы ошибок
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Прогноз')
plt.ylabel('Факт')
plt.title('Матрица ошибок')
plt.show()
```



Precision (точность) и recall (чувствительность)

Это фундаментальные метрики, используемые для оценки эффективности моделей классификации, которые дают представление о способности модели правильно определять положительные экземпляры и особенно полезны в сценариях, где классы несбалансированы.

Precision



Определение

- Измеряет долю положительных предсказаний, которые действительно верны.
- Отвечает на вопрос: «Сколько из всех случаев, которые были предсказаны как положительные, были действительно положительными?»

Формула

$$Precision = \frac{TP}{TP+FP}$$

где:

- Истинные положительные результаты (TP): Количество экземпляров, правильно предсказанных как положительные.
- Ложноположительные результаты (FP): количество экземпляров, неверно предсказанных как положительные.

Пример

Модель, которая классифицирует электронные письма как спам или не спам.

Модель делает следующие прогнозы:

- Истинные положительные результаты (TP): 50 (письма, правильно идентифицированные как спам)
- Ложноположительные результаты (FP): 10 (письма, неправильно идентифицированные как спам)

Точность можно рассчитать следующим образом:

$$Precision = \frac{50}{50+10} \approx 0.83$$

83% писем, предсказанных как спам, на самом деле оказались спамом.

Recall



Определение

- Измеряет долю реальных положительных случаев, которые были правильно определены моделью.
- Отвечает на вопрос: «Сколько из всех фактических положительных примеров было правильно предсказано как положительные?»

Формула

$$Recall = \frac{TP}{TP+FN}$$

где:

- True Positives (TP): Количество экземпляров, правильно предсказанных как положительные.
- Ложные негативы (FN): Количество экземпляров, неверно предсказанных как отрицательные.

Пример

Классификация электронной почты. Модель делает следующие прогнозы:

- Истинно положительные (TP): 50 (письма, правильно идентифицированные как спам).
- Ложноотрицательные (FN): 20 (письма, неправильно идентифицированные как не спам).

Чувствительность можно рассчитать следующим образом:

$$Recall = \frac{50}{50+20} \approx 0.71$$

71% реальных спам-писем были правильно идентифицированы моделью.

Высокий precision, низкий recall

Сценарий

Модель очень консервативна в составлении положительных прогнозов. Она делает положительные прогнозы только тогда, когда очень уверена в этом

Результат

Количество ложноположительных результатов (FP) низкое, но количество ложноотрицательных результатов (FN) высокое

Пример

В системе обнаружения спама модель с высокой точностью будет редко отмечать легитимные письма как спам (низкая FP), но может пропустить много настоящих спамных писем (высокая FN)

Высокий recall, низкий precision

Сценарий

Модель очень либерально относится к положительным прогнозам. Она предсказывает положительные результаты даже при низкой уверенности

Результат

Число ложноотрицательных результатов (FN) низкое, но число ложноположительных результатов (FP) высокое

Пример

В системе медицинской диагностики модель с высоким коэффициентом отзыва будет отлавливать почти все реальные случаи заболевания (низкий FN), но она также может отмечать наличие заболевания у многих здоровых людей (высокий FP)



Кривая «Precision-Recall»

Показывает соотношение precision (ось y) и recall (ось x) для различных пороговых значений. Помогает понять компромисс между precision и recall и выбрать оптимальное пороговое значение, исходя из конкретных требований задачи.

Ключевые моменты

Визуализация
компромисса

Площадь под кривой
(AUC)

Сравнение моделей

Пример



Кривая «Precision-Recall» на наборе данных Breast Cancer с алгоритмом Decision Tree.
Реализуем классификатор Decision Tree, чтобы предсказать, является ли опухоль рака молочной железы доброкачественной или злокачественной, используя набор данных Breast Cancer.
Построим кривую Precision-Recall, чтобы оценить эффективность модели.

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_recall_curve, precision_score, recall_score,
confusion_matrix, classification_report
from sklearn.datasets import load_breast_cancer
```


Шаг 2: Загрузка и подготовка набора данных

```
# Загрузить набор данных
data = load_breast_cancer()
X, y = data.data, data.target

# Преобразуем в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
```

Шаг 3: Разделение на обучение и тестирование

```
# Разделите данные на обучающий и тестовый наборы  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

Шаг 4: Обучение модели дерева решений

```
# Инициализируем классификатор дерева решений с максимальной глубиной 3
model = DecisionTreeClassifier(max_depth=3, random_state=42)

# Обучите модель на обучающих данных
model.fit(X_train, y_train)
```

Шаг 5: Предсказания и расчеты Precision и Recall

```
# Делаем предсказания на тестовом наборе
y_pred = model.predict(X_test)
y_scores = model.predict_proba(X_test)[:, 1]

# Рассчитайте precision и recall
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
print(f'Precision: {precision}')
print(f'Recall: {recall}')

# Вывести отчет о классификации
print('Отчет о классификации:')
print(classification_report(y_test, y_pred))

# Вывести матрицу ошибок
conf_matrix = confusion_matrix(y_test, y_pred)
print('Матрица ошибок:')
print(conf_matrix)

# Визуализация матрицы ошибок
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=data.target_names, yticklabels=data.target_names)
plt.xlabel('Прогноз')
plt.ylabel('Факт')
plt.title('Матрица ошибок')
plt.show()
```

Шаг 6: Постройте кривую «Precision-Recall»

```
# Рассчитайте кривую Precision-Recall
precisions, recalls, thresholds = precision_recall_curve(y_test, y_scores)

# Построить график кривой Precision-Recall
plt.plot(recalls, precisions, marker='.')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Кривая Precision-Recall')
plt.show()
```

Экспресс-опрос

?

Что такое F1-score?



F1-score

Это метрика, используемая для оценки эффективности модели классификации. Представляет собой среднее гармоническое значение precision и recall, обеспечивая единый показатель, который уравнивает обе метрики.

F1-score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- Истинноположительные результаты (TP): Количество экземпляров, правильно предсказанных как положительные.
- Ложноположительные результаты (FP): количество экземпляров, неверно предсказанных как положительные.
- Ложноотрицательные результаты (FN): Количество экземпляров, неверно предсказанных как отрицательные.

Пример

	Предсказанное положительное	Предсказанное отрицательное
Фактическое положительное	50	10
Фактическое отрицательное	5	35

Пример

Из матрицы ошибок можно определить следующие значения:

- Истинные положительные результаты (TP): 50
- Ложные положительные результаты (FP): 5
- Ложноотрицательные (FN): 10

Используя эти значения, можно рассчитать точность и чувствительность:

$$Precision = \frac{50}{50+5} \approx 0.91, Recall = \frac{50}{50+10} \approx 0.83$$

Можно рассчитать F1-score:

$$F1 \approx 2 \times \frac{0.91 \times 0.83}{0.91 + 0.83} \approx 0.87$$

F1-score для этой модели составляет примерно 0.87.

F1-Score



Преимущества

- Баланс между precision и recall
- Полезен для несбалансированных наборов данных
- Нечувствительность к распределению классов

Ограничения

- Не различает типы ошибок
- Единая метрика



ВОПРОСЫ





Цель машинного обучения

Это разработка моделей, которые хорошо обобщают новые, неизвестные данные. Достичь этого баланса бывает непросто, и модели могут страдать от двух распространенных проблем: переобучения и недообучения.

Причины переобучения

-  Сложные модели
-  Недостаточное количество обучающих данных
-  Шум в данных
-  Отсутствие регуляризации
-  Слишком много признаков

Причины недообучения



Простые модели



Недостаточное время обучения



Неадекватные признаки



Высокая регуляризация



Неправильный выбор модели

Стратегии предотвращения переобучения

Кросс-валидация

Регуляризация

Обрезка

Ранняя остановка

Ансамблевые методы

Стратегии предотвращения недообучения

Повышайте сложность
моделей

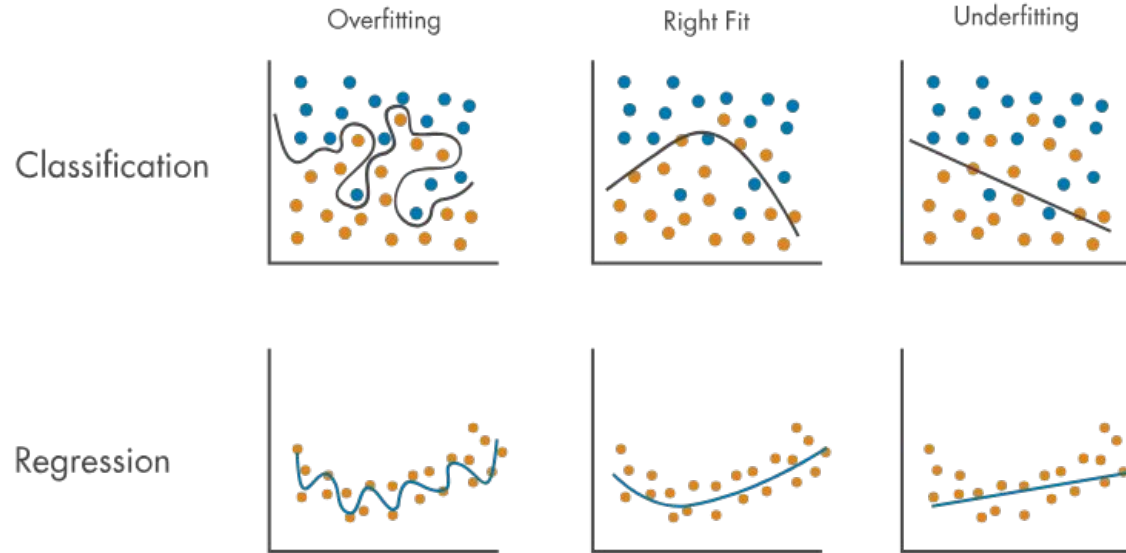
Инженерия признаков

Снижение регуляризации

Обучать дольше

Используйте подходящие
модели

Overfitting и underfitting





Кривые обучения

Это графические представления, иллюстрирующие, как меняется производительность модели с ростом опыта, обычно измеряемого количеством обработанных ею обучающих данных.

Избыточная и недостаточная подгонка с помощью кривых обучения



Переобучение

- Точность обучения высока и остается стабильной или даже растет.
- Точность проверки значительно ниже точности обучения и может даже снижаться со временем.
- Модель запоминает учебные данные, а не изучает общие закономерности.

Недообучение

- Точность обучения и проверки низкая и остается относительно постоянной.
- Модель не способна уловить существенные особенности данных, что приводит к низкой производительности на обоих наборах.



Функция `learning_curve` в Scikit-learn

Это инструмент для оценки производительности модели машинного обучения по мере увеличения размера обучающего набора данных.

Ключевые параметры: estimator



Тип

Объект

Описание

Модель машинного обучения или оценщик, который будет использоваться. Этот объект должен реализовывать методы fit и predict.

Ключевые параметры: X



Тип

array-like, форма (n_samples, n_features)

Описание

Входные данные, которые будут использоваться для обучения модели.

Ключевые параметры: y



Тип

array-like, форма (n_образцов) или (n_образцов, n_функций)

Описание

Целевые значения (метки классов в классификации, вещественные числа в регрессии).

Ключевые параметры: train_sizes



Тип

array-like, форма (n_ticks,)

Описание

Относительное или абсолютное количество обучающих примеров, которые будут использованы для построения кривой обучения. Если тип - float, оно рассматривается как дробь от максимального размера обучающего набора.

По умолчанию: `np.linspace(0.1, 1.0, 5)`

Ключевые параметры: cv



Тип

int, генератор кросс-валидации или итерируемая переменная

Описание

Определяет стратегию разбиения кросс-валидации. Возможными значениями для cv являются:

- None, чтобы использовать 5-кратную кросс-валидацию по умолчанию.
- Целое число, чтобы указать количество наборов.

Ключевые параметры: scoring



Тип

string, callable или None

Описание

Строка или вызываемый объект/функция scorer с сигнатурой `scorer(estimator, X, y)`. Если None, то используется оценщик по умолчанию (если он есть).

Пример



Диагностика переобучения и недообучения с помощью кривых обучения в Python

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve, train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.datasets import fetch_california_housing
```

Шаг 2: Загрузка и подготовка набора данных

```
# Загрузите набор данных
data = fetch_california_housing()
X, y = data.data, data.target

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Шаг 3: Определите функцию для построения кривых обучения

```
def plot_learning_curves(estimator, X, y, train_sizes=np.linspace(0.1, 1.0, 10), cv=5):
    train_sizes, train_scores, validation_scores = learning_curve(
        estimator, X, y, train_sizes=train_sizes, cv=cv, scoring='neg_mean_squared_error'
    )

    train_scores_mean = -train_scores.mean(axis=1)
    validation_scores_mean = -validation_scores.mean(axis=1)

    plt.figure(figsize=(12, 6))
    plt.plot(train_sizes, train_scores_mean, 'o-', color='r', label='Training error')
    plt.plot(train_sizes, validation_scores_mean, 'o-', color='g', label='Validation error')
    plt.xlabel('Размер обучающих данных')
    plt.ylabel('Ошибка')
    plt.title('Кривые обучения')
    plt.legend(loc='best')
    plt.grid()
    plt.show()
```

Шаг 4: Обучение модели и построение кривых обучения

```
# Инициализируем модель дерева решений
model = DecisionTreeRegressor(max_depth=5, random_state=42)

# Построить кривые обучения
plot_learning_curves(model, X_train, y_train)
```


Интерпретация кривых обучения

- Если кривые обучения показывают, что ошибка обучения постоянно уменьшается и приближается к нулю, а ошибка проверки остается значительно выше и может даже увеличиться, это свидетельствует о переобучении.
- Если кривые обучения показывают, что ошибки обучения и проверки высоки и остаются относительно постоянными, это свидетельствует о недообучении.
- Хорошо подогнанная модель имеет кривые обучения, на которых ошибки обучения и проверки уменьшаются и сходятся к одинаковому значению.

Экспресс-опрос



Что такое кросс-валидация?



Кросс-валидация

Это техника, используемая для оценки производительности модели машинного обучения и диагностики таких проблем, как переобучение и недообучение. Разбивая данные на несколько подмножеств и обучая модель на различных комбинациях этих подмножеств, кросс-валидация позволяет получить более полную оценку эффективности модели.

Пример



Диагностика переобучения и недообучения с помощью перекрестной валидации в Python

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import accuracy_score
```

Шаг 2: Загрузка и подготовка набора данных

```
# Загрузить набор данных
data = load_breast_cancer()
X, y = data.data, data.target

# Преобразуем в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
```

Шаг 3: Выполните кросс-валидацию

```
# Инициализируем модель дерева решений
model = DecisionTreeClassifier(random_state=42)

# Выполняем 5-кратную перекрестную валидацию
cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')

# Выведите оценки кросс-валидации
print(f'Cross-Validation Scores: {cv_scores}')
print(f'Средняя оценка кросс-валидации: {cv_scores.mean()}')
```

Шаг 4: Анализ результатов

```
# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Обучите модель на обучающих данных
model.fit(X_train, y_train)

# Делаем предсказания на обучающем и тестовом наборах
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Рассчитайте точность на обучающем и тестовом наборах
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f'Training Accuracy: {train_accuracy}')
```


Экспресс-опрос

?

Что такое регуляризация?



Регуляризация

Это техника, используемая в машинном обучении для предотвращения чрезмерной подгонки путем добавления штрафа к сложности модели. Этот штраф не позволяет модели подстраиваться под шум в обучающих данных, тем самым улучшая ее обобщение на новые, неизвестные данные.

Методы регуляризации

Lasso (L1) регрессия

Ridge (L2) регрессия



Цель регуляризации

Снизить сложность модели за счет штрафов за большие коэффициенты.

Регрессия LASSO



Определение

Регрессия LASSO или регуляризация L1, добавляет штраф, равный абсолютному значению коэффициентов, к функции стоимости.

Формула

$$\text{Функция потерь} = \frac{1}{2} \sum (y - \hat{y})^2 + \lambda \sum |w|,$$

где:

- y - фактическое значение,
- $\hat{y}$ - предсказанное значение,
- w обозначает коэффициенты признаков,
- λ - параметр регуляризации, который управляет силой штрафа.

Ридж-Регрессия



Определение

Ридж-регрессия или регуляризация L2, добавляет к функции потерь штраф, равный квадрату коэффициентов.

Формула

$$\text{Функция потерь} = \frac{1}{2} \sum (y - \hat{y})^2 + \lambda \sum w^2$$

где:

- y - фактическое значение,
- $\hat{y}$ - предсказанное значение,
- w обозначает коэффициенты признаков,
- λ - параметр регуляризации, который управляет силой штрафа.

Влияние на модель



Уменьшение перебора



Включение всех признаков



Улучшенная стабильность

Сравнение гребневой регрессии и регрессии LASSO



Штрафной компонент



Выбор признаков



Примеры использования

Пример



Применим регрессию LASSO и ридж-регрессию, используя набор данных California Housing.
Этот набор данных содержит информацию о различных характеристиках домов
в Калифорнии и соответствующих им ценах

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Lasso, Ridge
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import fetch_california_housing
```

Шаг 2: Загрузка и подготовка набора данных

```
# Загрузка набора данных
data = fetch_california_housing()
X, y = data.data, data.target

# Преобразование в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
```

Шаг 3: Разделение на обучение и тестирование

```
# Разделите данные на обучающий и тестовый наборы  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

Шаг 4: Стандартизация данных

```
# Инициализация стандартного скейлера
scaler = StandardScaler()

# Подгонка и преобразование обучающих данных
X_train_scaled = scaler.fit_transform(X_train)

# Преобразование тестовых данных
X_test_scaled = scaler.transform(X_test)
```

Шаг 5: Обучение и оценка модели регрессии LASSO

```
# Инициализируем регрессионную модель LASSO с параметром регуляризации alpha
lasso = Lasso(alpha=0.1)

# Обучите модель на обучающих данных
lasso.fit(X_train_scaled, y_train)

# Делаем прогнозы на тестовых данных
y_pred_lasso = lasso.predict(X_test_scaled)

# Оценить модель
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print(f'Регрессия LASSO - средняя квадратичная ошибка: {mse_lasso}')
print(f'Регрессия LASSO - R-квадрат: {r2_lasso}')
```

Шаг 6: Обучение и оценка модели гребневой регрессии

```
# Инициализируем модель ридж-регрессии с параметром регуляризации alpha
ridge = Ridge(alpha=0.1)

# Обучить модель на обучающих данных
ridge.fit(X_train_scaled, y_train)

# Делаем прогнозы на тестовых данных
y_pred_ridge = ridge.predict(X_test_scaled)

# Оценить модель
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print(f'Ridge Regression - Mean Squared Error: {mse_ridge}')
print(f'Ridge Regression - R-квадрат: {r2_ridge}')
```

Шаг 7: Сравнение результатов

```
# Сравниваем коэффициенты регрессии LASSO и гребневой регрессии
coefficients = pd.DataFrame({
    'Feature': data.feature_names,
    'Коэффициенты LASSO': lasso.coef_,
    'Ridge Coefficients': ridge.coef_
})

print(coefficients)
```




ВОПРОСЫ





ЗАДАНИЕ



Задание

Завершить работу над задачами с практикумов 19 и 20.



Подготовка к заданиям



Скачайте [датасет](#) с данными о погоде в городе N в течение года. Датасет содержит данные о температуре воздуха, температуре воды, скорости ветра и наличии осадков за каждый день года.

Задание 1



Разделите датасет на тестовую и обучающую выборку. Постройте линейную регрессию между температурой воздуха и воды. Визуализируйте зависимость и вычислите показатели качества регрессии.

Задание 2



Добавьте в регрессию еще один параметр - скорость ветра. Постройте линейную регрессию и вычислите ее характеристики.

Задание 3



В предыдущий код добавьте третий объясняющий параметр – дождь. Какая из регрессий оказалась самой точной?

Задание 4



Постройте регрессию, в которой объясняемая переменная – дождь, а объясняющие – все остальные. Вычислите ее точность

Задание 5



Теперь для той же задачи постройте дерево решений. Вычислите его точность.

Задание 6



Постройте модель, предсказывающую дождь, с помощью K ближайших соседей. Вычислите ее точность. Попробуйте подобрать наиболее удачное K .

Задание 7



Используйте набор данных California Housing, чтобы предсказать цены на дома на основе различных характеристик.

Задание 8



Используйте набор данных Mtcars, чтобы предсказать количество миль на галлон (mpg) для автомобилей на основе таких характеристик, как мощность, вес и количество цилиндров.

Задание 9



Используйте набор данных Breast Cancer, чтобы предсказать, является ли опухоль злокачественной или доброкачественной, основываясь на различных признаках.

Задание 10



Используйте набор данных Pima Indians Diabetes, чтобы предсказать наличие диабета у пациента на основе таких характеристик, как уровень глюкозы, артериальное давление и индекс массы тела.

Задание 11



Используйте набор данных Digits для предсказания рукописных цифр на основе значений пикселей.

Задание 1



Реализуйте линейную регрессию для прогнозирования цен на жилье с помощью набора данных California Housing. Оцените эффективность модели, используя среднюю квадратичную ошибку (MSE) и R-квадрат (R^2). Постройте график остатков, чтобы проверить наличие каких-либо закономерностей.

Задание 2



Реализуйте логистическую регрессию для классификации злокачественной или доброкачественной опухоли на наборе данных Breast Cancer. Оцените модель с помощью показателей accuracy, precision, recall и F1-score.

Задание 3



Реализуйте классификатор на основе дерева решений для предсказания качества вина, используя набор данных Wine. Оцените модель с помощью перекрестной валидации и постройте дерево решений.

Задание 4



Реализуйте модель K-Nearest Neighbors (KNN) для классификации рукописных цифр с помощью набора данных Digits. Оцените модель по точности и постройте матрицу ошибок.

Задание 5



Постройте кривые обучения для диагностики переобучения и недообучения, используя набор данных Iris и модель Decision Tree. Постройте кривые обучения для ошибок обучения и проверки.

Задание 6



Реализуйте регрессию LASSO и ридж-регрессию, используя набор данных Iris. Сравните эффективность обеих моделей, используя среднюю квадратичную ошибку (MSE) и R-квадрат (R^2). Постройте графики коэффициентов обеих моделей.

Задание 7



Обучите классификатор дерево решений на наборе данных Iris. Постройте кривую Precision-Recall и обсудите компромисс между точностью и чувствительностью. Для простоты преобразуйте многоклассовую задачу в задачу бинарной классификации.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Используйте набор данных California Housing, чтобы сравнить эффективность линейной регрессии и деревьев решений при прогнозировании цен на жилье.
2. Используйте набор данных Wine, чтобы предсказать качество вина на основе его химических свойств. Выберите две модели из пройденных и сравните их эффективность.

Домашнее задание

1. Обучите модель логистической регрессии на наборе данных Breast Cancer. Оцените модель с помощью показателей точности, precision, recall и F1-score. Постройте кривую Precision-Recall.
2. Проведите кросс-валидацию с использованием [набора данных](#) (таргет - последний столбец) и модели логистической регрессии. Оцените модель с помощью оценок кросс-валидации и постройте кривую обучения.



ВОПРОСЫ



Заключение

